

# Reviewer Pack — Secant Variety Dimension Lower Bounds for Disjointness Commu...

Entry adedf0b3ac89 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 14:52:22 UTC

## Secant Variety Dimension Lower Bounds for Disjointness Communication Complexity

**Entry ID:** adedf0b3ac89 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 14:52:22 UTC

### 1. Conjecture

**Field A** (mathematical branch): ALGEBRAIC\_GEOMETRY\_OF\_SECANT\_VARIETIES **Field B** (complexity object): COMMUNICATION\_COMPLEXITY

#### Statement:

Let  $M$  be the communication matrix of the disjointness function  $\text{DISJ}_n$ . Define  $\tau(M)$  as the dimension of the secant variety of the variety  $V(M)$  spanned by the rank-1 tensors of  $M$ . Then  $\tau(M) \geq \Omega(n)$ , and equality holds for all  $n \leq 40$ .

#### Rationale (proposer's reasoning):

Secant variety dimensions capture the minimal number of rank-1 tensors needed to approximate a matrix, which directly relates to communication complexity. For  $\text{DISJ}_n$ , the secant variety's dimension grows linearly with  $n$ , matching the  $\Omega(n)$  lower bound. This connects algebraic geometry to communication complexity via tensor rank theory.

**Taxonomy category:** BARRINGTON\_ALG (status at proposal time: partially\_alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 86f71c0add7e62a5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

default:  $\geq 80\%$  seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| NATURAL_PROOFS | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |

| Barrier       | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|---------------|---------|------------|------------------|------------------|
| ALGEBRIZATION | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON   | SAFE    | 0.95       | UNCERTAIN        | SAFE             |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    m, k = len(A), len(B[0])
    n = len(B)
    C = [[0 for _ in range(k)] for _ in range(m)]
    for i in range(m):
        for j in range(k):
            for l in range(n):
                C[i][j] += A[i][l] * B[l][j]
    return C

def transpose(A):
    m, n = len(A), len(A[0])
    B = [[0 for _ in range(m)] for _ in range(n)]
    for i in range(m):
        for j in range(n):
            B[j][i] = A[i][j]
    return B

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    rref = [[A[i][j] for j in range(n)] for i in range(m)]
    lead = 0
    for r in range(m):
        if lead >= n:
            break
        i = r
        while rref[i][lead] == 0:
            i += 1
```

```

        if i == m:
            i = r
            lead += 1
            if lead == n:
                break
        rref[r], rref[i] = rref[i], rref[r]
        factor = rref[r][lead]
        for j in range(n):
            rref[r][j] /= factor
        for i in range(m):
            if i != r and rref[i][lead]:
                factor = rref[i][lead]
                for j in range(n):
                    rref[i][j] -= factor * rref[r][j]
        lead += 1
    return rref

def rank(A):
    rref = gaussian_elimination(A)
    rank = sum(1 for row in rref if any(row))
    return rank

def secant_variety_dimension(M):
    m, n = len(M), len(M[0])
    rank_M = rank(M)
    if rank_M == 1:
        return 1
    A_augmented = [[M[i][j] for j in range(n)] + [1] for i in range(m)]
    rref_A_augmented = gaussian_elimination(A_augmented)
    k = len(rref_A_augmented[0]) - 1
    tau_M = m * n - rank_M * (k - 1) - rank_M
    return tau_M

def disjointness_matrix(n):
    M = [[0 for _ in range(2**n)] for _ in range(2**n)]
    for i in range(2**n):
        for j in range(2**n):
            if (i & j) == 0:
                M[i][j] = 1
    return M

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        M = disjointness_matrix(n)
        tau_M = secant_variety_dimension(M)
        if tau_M < n / 2:
            return {
                "metric_name": "secant_variety_dimension",
                "metric_value": tau_M,
                "instances_tested": len(n_values),
                "conjecture_holds": False,
                "counterexample": f"n={n}, tau(M)={tau_M}"
            }

```

```

        results.append(tau_M)
    mean = sum(results) / len(results)
    std = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
    return {
        "metric_name": "secant_variety_dimension",
        "metric_value": mean,
        "instances_tested": len(n_values),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = primes[:30]
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}, \"instances_tested\": {result['instances_tested']}, \"conjecture_holds\": {result['conjecture_holds']}, \"counterexample\": \"{result['counterexample']}\"}}")
    mean = sum(result['metric_value'] for result in results) / len(results)
    std = math.sqrt(sum((result['metric_value'] - mean) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result['conjecture_holds']) / len(results)
    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(not result['conjecture_holds'] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

ic_name": "secant_variety_dimension", "metric_value": 0, "instances_tested": 6, "conjecture_holds":
TRIAL: {"seed": 547, "metric_name": "secant_variety_dimension", "metric_value": 0, "instances_tested":
TRIAL: {"seed": 593, "metric_name": "secant_variety_dimension", "metric_value": 0, "instances_tested":
TRIAL: {"seed": 631, "metric_name": "secant_variety_dimension", "metric_value": 0, "instances_tested":
TRIAL: {"seed": 677, "metric_name": "secant_variety_dimension", "metric_value": 0, "instances_tested":
TRIAL: {"seed": 727, "metric_name": "secant_variety_dimension", "metric_value": 0, "instances_tested":
TRIAL: {"seed": 773, "metric_name": "secant_variety_dimension", "metric_value": 0, "instances_tested":
TRIAL: {"seed": 821, "metric_name": "secant_variety_dimension", "metric_value": 0, "instances_tested":
TRIAL: {"seed": 877, "metric_name": "secant_variety_dimension", "metric_value": 0, "instances_tested":
TRIAL: {"seed": 929, "metric_name": "secant_variety_dimension", "metric_value": 0, "instances_tested":

```

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_204982c5.py", line 130, in <module>
    mean = sum(result['metric_value'] for result in results) / len(results)
            ^^^^^^

```

NameError: name 'results' is not defined. Did you mean: 'result'?

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test crashed due to undefined variable 'results', preventing reliable evaluation of conjecture support. | next: Fix the test code's results handling and re-run experiments with proper validation

## 11. Audit log (LLM calls)

**Total LLM calls:** 8

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 68881        |
| 2 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 51091        |
| 3 | preregistration | ollama_remote | qwen3:8b         | 0         | 0   | 23906        |
| 4 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 20489        |
| 5 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 15762        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 20107        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15048        |
| 8 | judge           | ollama_remote | qwen3:8b         | 0         | 0   | 16200        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 231484 ms total latency. Provider mix: {'ollama\_remote': 8}

*(full prompt+response transcripts available in research/audit/adedf0b3ac89.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/adedf0b3ac89.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/adedf0b3ac89.tar.gz (if generated)